



Министерство науки и высшего образования Российской Федерации
Федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ

«Фундаментальные Науки»

КАФЕДРА

ФН-12 «Математическое моделирование»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ НА ТЕМУ:

Перемножение матриц

Студент:

Подобедов В. В.

дата, подпись

Ф.И.О.

Преподаватель:

Волкова Л. Л.

дата, подпись

Ф.И.О.

Москва, 2023

Содержание

Введение	3
1 Аналитическая часть	4
2 Конструкторская часть	5
2.1 Умножение матриц классическим способом	5
2.2 Метод Винограда	5
2.3 Метод Штрассена	6
2.4 Функции программы	6
3 Технологическая часть	8
4 Исследовательская часть	16
Заключение	22
Список используемых источников	23

Введение

Цель лабораторной работы: провести сравнительный анализ алгоритмов умножения матриц.

Для достижения поставленной цели требуется решить следующие **задачи**.

1. Описать три алгоритма умножения матриц — классический, Винограда и Штрассена.
2. Реализовать разработанные алгоритмы.
3. Выполнить тестирование реализации разработанных алгоритмов.
4. Дать оценку сложности алгоритмов.
5. Оценить эффективность каждого алгоритма (по памяти и времени).

1 Аналитическая часть

Матрица — математический объект, записываемый в виде прямоугольной таблицы, который представляет собой совокупность строк и столбцов, на пересечении которых находятся его элементы. Количество строк и столбцов задает размер матрицы.

Классический алгоритм умножения матриц — это стандартный метод перемножения матриц в линейной алгебре. Если есть две матрицы A размером $m \times n$ и B размером $n \times p$, то произведение матриц $C = AB$ будет матрицей размером $m \times p$. Каждый элемент $C[i][j]$ матрицы C вычисляется путем умножения элементов соответствующей строки матрицы A на соответствующие элементы столбца матрицы B и последующим сложением результатов.

Алгоритм Винограда для умножения матриц [1] — это метод, где предварительно вычисляются некоторые значения и используются для уменьшения количества умножений в основном цикле умножения матриц. Этот метод полезен для больших матриц, где вычислительные затраты могут быть значительными.

Алгоритм Штрассена для умножения матриц [1] — это метод умножения матриц, который уменьшает количество операций умножения стандартным способом. Он основан на разделении матриц на подматрицы меньшего размера, что позволяет снизить сложность умножения. Основная идея алгоритма заключается в том, что вместо классического умножения матриц A и B размерности $n \times n$ можно разбить каждую из них на четыре подматрицы размера $\frac{n}{2} \times \frac{n}{2}$, что дает 8 уравнений для вычисления произведения. Однако алгоритм Штрассена использует не все эти уравнения, а только 7, чтобы сократить количество операций умножения.

2 Конструкторская часть

2.1 Умножение матриц классическим способом

Формула для вычисления элемента $C[i][j]$ выглядит следующим образом (1):

$$C_{ij} = \sum_{k=1}^n A_{ik} \cdot B_{kj} \quad (1)$$

Нетрудно понять, что в данной формуле i — номер строки, j — номер столбца, k — индекс суммирования, C — результирующая матрица, A — первая матрица, B — вторая матрица. Это означает, что каждый элемент $C[i, j]$ матрицы C равен сумме произведений соответствующих элементов строки i матрицы A и столбца j матрицы B . Процесс вычисления всех элементов матрицы C повторяется для каждой пары (i, j) в пределах размерности матрицы C .

2.2 Метод Винограда

Алгоритм Винограда включает следующие шаги.

1. Создать новую матрицу C размерностью $m \times n$ и заполнить ее нулями.
2. Создать вспомогательные массивы row_factor и col_factor для оптимизации вычислений. Их размерность равна числу строк матрицы A и числу столбцов матрицы B соответственно.
3. Вычислить $row_factor[i]$ как сумму произведений пар элементов из $A[i]$, умноженных на соответствующие элементы из $A[i]$.
4. Вычислите $col_factor[j]$ как сумму произведений пар элементов из $B[j]$, умноженных на соответствующие элементы из $B[j]$.
5. Используя предварительно вычисленные row_factor и col_factor , вычислите каждый элемент матрицы C следующим образом:

$$C[i][j] = rc_f + \sum_{k=0}^{k-1} (A[i][k] + B[k][j]) \cdot (A[i][k+1] + B[k+1][j]), \text{ где}$$

$$rc_f = -row_factor[i] - col_factor[j]$$

6. Если размерность матрицы нечетна, нужно добавить к соответствующему элементу C произведение соответствующих элементов из A и B , которые не участвовали в основных вычислениях.

2.3 Метод Штрассена

Шаги алгоритма Штрассена, следующие.

1. Каждая из матриц A и B разбивается на четыре части размера $\frac{n}{2} \times \frac{n}{2}$. Таким образом получаются такие матрицы: $\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}$ и $\begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}$
2. Вычисляются промежуточные матрицы от M_1 до M_7 по формулам:

$$M_1 = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22}) \cdot B_{11}$$

$$M_3 = A_{11} \cdot (B_{12} - B_{22})$$

$$M_4 = A_{22} \cdot (B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12}) \cdot B_{22}$$

$$M_6 = (A_{21} - A_{11}) \cdot (B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$$

3. Используя промежуточные матрицы M_1 до M_7 , вычисляются четыре части результирующей матрицы C :

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

4. Полученные части результирующей матрицы C_{11} , C_{12} , C_{21} , C_{22} объединяются в единую матрицу C .

2.4 Функции программы

Требуется разработать следующие функции.

- Функция “matrixAddition” — функция, которая складывает матрицы.
- Функция “matrixSubtraction” — функция, которая перемножает две матрицы.
- Функция “strassenMatrixMultiply” — функция для умножения матриц методом Штрассена.
- Функция “classicMatrixMultiply” — функция для умножения матриц классическим способом.

- Функция “matrixMultiplyVinograd” — Функция для умножения матриц с использованием алгоритма Винограда.
- Функция “printMatrix” — функция для вывода матрицы.

К программе предъявлены следующие требования. В начале выполнения работы программы пользователь выбирает каким методом будут перемножены матрицы. Далее с помощью выбранного алгоритма вычисляется произведение матриц и на экран консоли выводится результирующая матрица.

3 Технологическая часть

Лабораторная работа была реализована на языке программирования C++. Кроме стандартной библиотеки `<iostream>`, которая предоставляет возможность пользоваться средствами ввода-вывода для стандартной консоли, была подключена библиотека `<vector>`. Она включает в себя шаблон класса для контейнеров последовательности. Вектор хранит элементы заданного типа в линейном расположении и обеспечивает быстрый случайный доступ к любому элементу. Также для вычисления работы программы была заимствована библиотека `<ctime>`.

Далее приведен листинг реализации трех алгоритмов (листинг 1).

Листинг 1 – Исходный код программы для нахождения произведения матриц разными методами

```
#include <iostream>
#include <vector>
#include <ctime>

using namespace std;

// Функция для сложения матриц
vector<vector<int>> matrixAddition(const vector<vector<int>>& A, const
    vector<vector<int>>& B) {
    int n = A.size();
    vector<vector<int>> result(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            result[i][j] = A[i][j] + B[i][j];
        }
    }

    return result;
}

// Функция для вычитания матриц
vector<vector<int>> matrixSubtraction(const vector<vector<int>>& A, const
    vector<vector<int>>& B) {
    int n = A.size();
    vector<vector<int>> result(n, vector<int>(n, 0));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            result[i][j] = A[i][j] - B[i][j];
        }
    }
}
```

```

        }
    }

    return result;
}

// Функция для умножения матриц методом Штрассена
vector<vector<int>> strassenMatrixMultiply(const vector<vector<int>>& A,
    const vector<vector<int>>& B) {
    int n = A.size();

    if (n <= 2) {
        vector<vector<int>> result(n, vector<int>(n, 0));
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                for (int k = 0; k < n; k++) {
                    result[i][j] += A[i][k] * B[k][j];
                }
            }
        }

        return result;
    }

    int newSize = n;
    if (n % 2 != 0) {
        newSize++;
    }

    vector<vector<int>> newA(newSize, vector<int>(newSize, 0));
    vector<vector<int>> newB(newSize, vector<int>(newSize, 0));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            newA[i][j] = A[i][j];
            newB[i][j] = B[i][j];
        }
    }

    vector<vector<int>> A11(newSize / 2, vector<int>(newSize / 2, 0))
        ;
    vector<vector<int>> A12(newSize / 2, vector<int>(newSize / 2, 0))

```

```

        ;
vector<vector<int>> A21(newSize / 2, vector<int>(newSize / 2, 0))
        ;
vector<vector<int>> A22(newSize / 2, vector<int>(newSize / 2, 0))
        ;
vector<vector<int>> B11(newSize / 2, vector<int>(newSize / 2, 0))
        ;
vector<vector<int>> B12(newSize / 2, vector<int>(newSize / 2, 0))
        ;
vector<vector<int>> B21(newSize / 2, vector<int>(newSize / 2, 0))
        ;
vector<vector<int>> B22(newSize / 2, vector<int>(newSize / 2, 0))
        ;
for (int i = 0; i < newSize / 2; i++) {
    for (int j = 0; j < newSize / 2; j++) {
        A11[i][j] = newA[i][j];
        A12[i][j] = newA[i][j + newSize / 2];
        A21[i][j] = newA[i + newSize / 2][j];
        A22[i][j] = newA[i + newSize / 2][j + newSize /
            2];

        B11[i][j] = newB[i][j];
        B12[i][j] = newB[i][j + newSize / 2];
        B21[i][j] = newB[i + newSize / 2][j];
        B22[i][j] = newB[i + newSize / 2][j + newSize /
            2];
    }
}

vector<vector<int>> M1 = strassenMatrixMultiply(matrixAddition(
    A11, A22), matrixAddition(B11, B22));
vector<vector<int>> M2 = strassenMatrixMultiply(matrixAddition(
    A21, A22), B11);
vector<vector<int>> M3 = strassenMatrixMultiply(A11,
    matrixSubtraction(B12, B22));
vector<vector<int>> M4 = strassenMatrixMultiply(A22,
    matrixSubtraction(B21, B11));
vector<vector<int>> M5 = strassenMatrixMultiply(matrixAddition(
    A11, A12), B22);
vector<vector<int>> M6 = strassenMatrixMultiply(matrixSubtraction
    (A21, A11), matrixAddition(B11, B12));
vector<vector<int>> M7 = strassenMatrixMultiply(matrixSubtraction

```

```

        (A12, A22), matrixAddition(B21, B22));

vector<vector<int>> C11 = matrixAddition(matrixSubtraction(
    matrixAddition(M1, M4), M5), M7);
vector<vector<int>> C12 = matrixAddition(M3, M5);
vector<vector<int>> C21 = matrixAddition(M2, M4);
vector<vector<int>> C22 = matrixAddition(matrixAddition(
    matrixSubtraction(M1, M2), M3), M6);

vector<vector<int>> result(newSize, vector<int>(newSize));
for (int i = 0; i < newSize / 2; i++) {
    for (int j = 0; j < newSize / 2; j++) {
        result[i][j] = C11[i][j];
        result[i][j + newSize / 2] = C12[i][j];
        result[i + newSize / 2][j] = C21[i][j];
        result[i + newSize / 2][j + newSize / 2] = C22[i
            ][j];
    }
}

// Матрица без лишних нулей
vector<vector<int>> finalResult(n, vector<int>(n));
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        finalResult[i][j] = result[i][j];
    }
}

return finalResult;
}

// Функция для умножения матриц классическим способом
vector<vector<int>> classicMatrixMultiply(const vector<vector<int>>& matA
    , const vector<vector<int>>& matB) {

    if (matA[0].size() != matB.size()) {
        cout << "Невозможно умножить матрицы : неподходящие размеры " <<
            endl;
        return {};
    }

    vector<vector<int>> result(matA.size(), vector<int>(matB[0].size

```

```

        ( ), 0));

    for (int i = 0; i < matA.size(); i++) {
        for (int j = 0; j < matB[0].size(); j++) {
            for (int k = 0; k < matA[0].size(); k++) {
                result[i][j] += matA[i][k] * matB[k][j];
            }
        }
    }

    return result;
}

// Функция для умножения матриц с использованием алгоритма Винограда
vector<vector<int>> matrixMultiplyVinograd(const vector<vector<int>>&
    matA, const vector<vector<int>>& matB) {
    int rowsA = matA.size();
    int colsA = matA[0].size();
    int rowsB = matB.size();
    int colsB = matB[0].size();

    if (colsA != rowsB) {
        cout << "Невозможно умножить матрицы : неподходящие размеры " <<
            endl;
        return {};
    }

    vector<vector<int>> result(rowsA, vector<int>(colsB, 0));

    // Предварительные вычисления для оптимизации
    vector<int> rowFactor(rowsA, 0);
    vector<int> colFactor(colsB, 0);
    for (int i = 0; i < rowsA; i++) {
        for (int k = 0; k < colsA - 1; k += 2) {
            rowFactor[i] += matA[i][k] * matA[i][k + 1];
        }
    }
    for (int j = 0; j < colsB; j++) {
        for (int k = 0; k < rowsB - 1; k += 2) {
            colFactor[j] += matB[k][j] * matB[k + 1][j];
        }
    }
}

```

```

// Основнойциклумножениясиспользованиемпредварительныхвычислений
for (int i = 0; i < rowsA; i++) {
    for (int j = 0; j < colsB; j++) {
        result[i][j] = -rowFactor[i] - colFactor[j];
        for (int k = 0; k < colsA - 1; k += 2) {
            result[i][j] += (matA[i][k] + matB[k +
                1][j]) * (matA[i][k + 1] + matB[k][j])
            ;
        }
        if (colsA % 2 != 0) {
            result[i][j] += matA[i][colsA - 1] * matB
                [rowsB - 1][j];
        }
    }
}

return result;
}

// Функциядлявыводаматрицы
void printMatrix(const vector<vector<int>>& mat) {
    for (const auto& row : mat) {
        for (int val : row) {
            cout << val << " ";
        }
        cout << endl;
    }
}

int main(){
    setlocale(0, "");
    cout << "Каким методомВыхотитеперемножитьматрицы    ?" << endl;
    cout << "Классический метод - 1" << endl;
    cout << "Метод Винограда - 2" << endl;
    cout << "Метод Штрассена - 3" << endl;
    int choose = 0;
    double medtime = 0;
    cin >> choose;

    if (choose == 1) {

```

```

vector<vector<int>> matrixA = { {1, 2, 3}, {4, 5, 6}, {7,
    8, 9} };
vector<vector<int>> matrixB = { {9, 8, 7}, {6, 5, 4}, {3,
    2, 1} };

cout << "Матрица A:" << endl;
printMatrix(matrixA);
cout << endl;

cout << "Матрица B:" << endl;
printMatrix(matrixB);
cout << endl;

clock_t start_time = clock();
vector<vector<int>> result = classicMatrixMultiply(
    matrixA, matrixB);
clock_t end_time = clock();
double cpu_time = (double(end_time - start_time)) /
    CLOCKS_PER_SEC;
medtime += cpu_time;
cout << "Результат умножения классическим методом : " << endl;
printMatrix(result);
cout << "Процессорное время: " << cpu_time << endl;
cout << endl;
}
else if (choose == 2) {

vector<vector<int>> matrixA = { {1, 2, 3}, {4, 5, 6}, {7,
    8, 9} };
vector<vector<int>> matrixB = { {9, 8, 7}, {6, 5, 4}, {3,
    2, 1} };

cout << "Матрица A:" << endl;
printMatrix(matrixA);
cout << endl;

cout << "Матрица B:" << endl;
printMatrix(matrixB);
cout << endl;

clock_t start_time = clock();
vector<vector<int>> result = matrixMultiplyVinograd(

```

```

        matrixA, matrixB);
clock_t end_time = clock();
double cpu_time = (double(end_time - start_time)) /
    CLOCKS_PER_SEC;
medtime += cpu_time;
cout << "Результат умножения с помощью алгоритма Винограда" << endl;
printMatrix(result);
cout << "Процессорное время: " << cpu_time << endl;
cout << endl;
}
else if (choose == 3) {
    vector<vector<int>> matrixA = { {1, 2, 3}, {4, 5, 6}, {7,
        8, 9} };
    vector<vector<int>> matrixB = { {9, 8, 7}, {6, 5, 4}, {3,
        2, 1} };
    cout << "Матрица A:" << endl;
    printMatrix(matrixA);
    cout << endl;
    cout << "Матрица B:" << endl;
    printMatrix(matrixB);
    cout << endl;

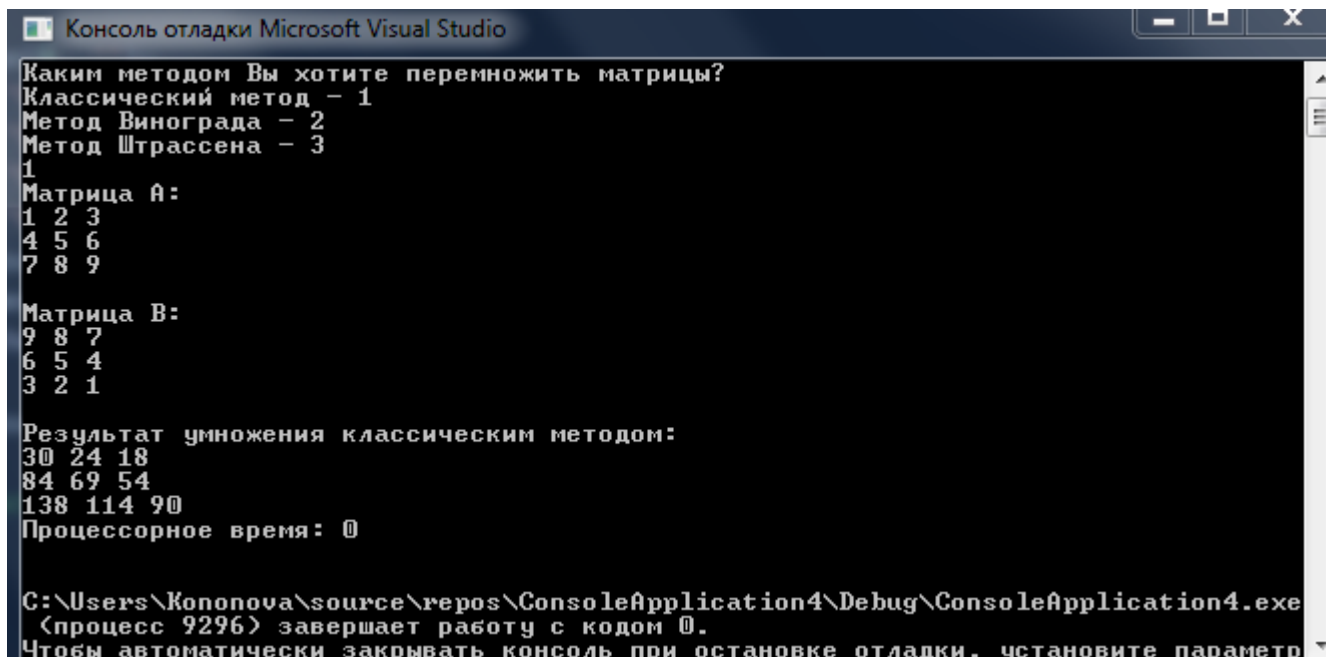
    clock_t start_time = clock();
    vector<vector<int>> result = strassenMatrixMultiply(
        matrixA, matrixB);
    clock_t end_time = clock();
    double cpu_time = (double(end_time - start_time)) /
        CLOCKS_PER_SEC;
    medtime += cpu_time;
    cout << "Результат умножения с помощью алгоритма Штрассена" << endl;
    printMatrix(result);
    cout << "Процессорное время: " << cpu_time << endl;
    cout << endl;
}
else {
    cout << "Неверный выбор" << endl;
}
return 0;
}

```

4 Исследовательская часть

Далее на рисунках представлен результат работы программы.

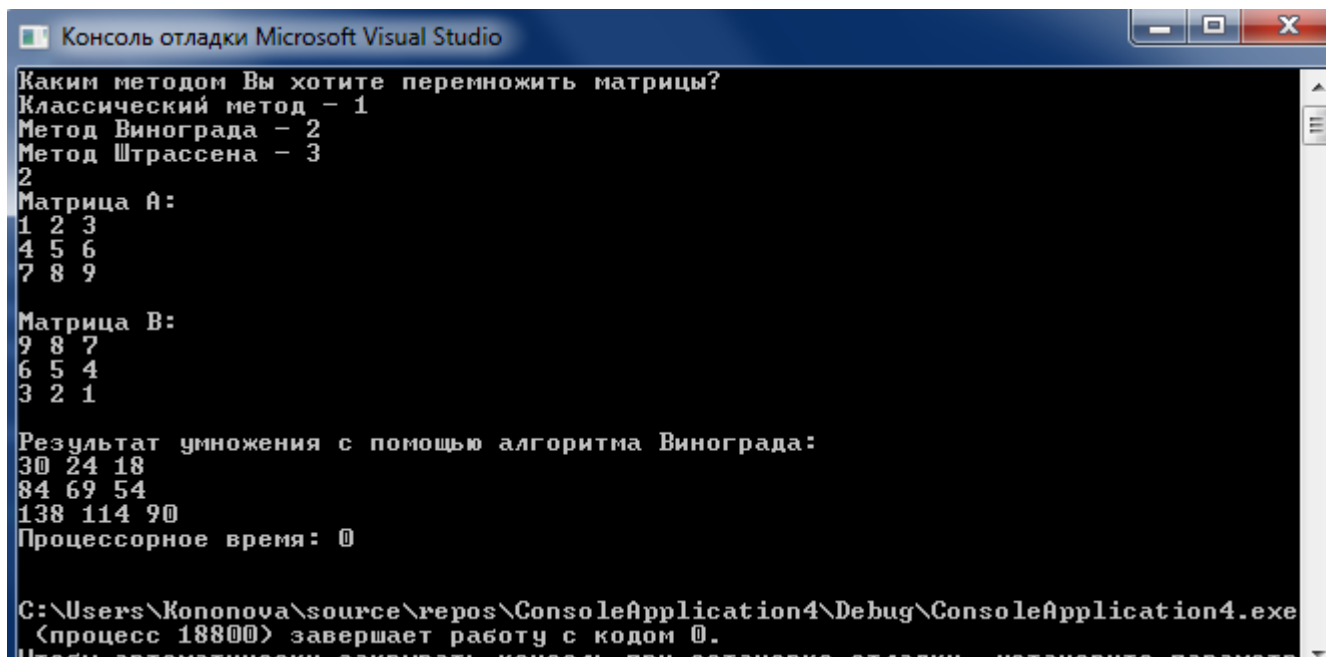
Результат работы программы умножения матриц 3×3 классическим способом, приведен на рис. 1.



```
Консоль отладки Microsoft Visual Studio
Каким методом Вы хотите перемножить матрицы?
Классический метод - 1
Метод Винограда - 2
Метод Штрассена - 3
1
Матрица A:
1 2 3
4 5 6
7 8 9
Матрица B:
9 8 7
6 5 4
3 2 1
Результат умножения классическим методом:
30 24 18
84 69 54
138 114 90
Процессорное время: 0
C:\Users\Kononova\source\repos\ConsoleApplication4\Debug\ConsoleApplication4.exe
(процесс 9296) завершает работу с кодом 0.
Чтобы автоматически закрывать консоль при остановке отладки, установите параметр
```

Рис. 1 – Пример №1

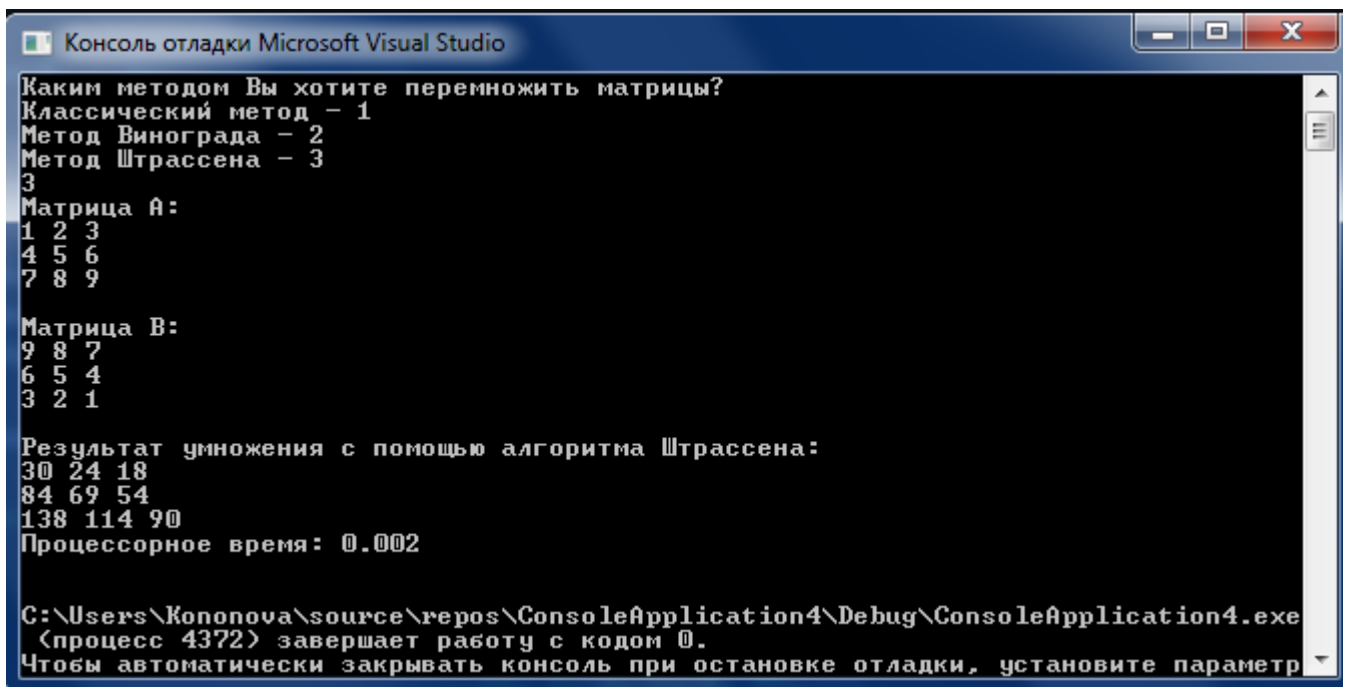
Результат работы программы умножения матриц 3×3 методом Винограда, приведен на рис. 2.



```
Консоль отладки Microsoft Visual Studio
Каким методом Вы хотите перемножить матрицы?
Классический метод - 1
Метод Винограда - 2
Метод Штрассена - 3
2
Матрица A:
1 2 3
4 5 6
7 8 9
Матрица B:
9 8 7
6 5 4
3 2 1
Результат умножения с помощью алгоритма Винограда:
30 24 18
84 69 54
138 114 90
Процессорное время: 0
C:\Users\Kononova\source\repos\ConsoleApplication4\Debug\ConsoleApplication4.exe
(процесс 18800) завершает работу с кодом 0.
Чтобы автоматически закрывать консоль при остановке отладки, установите параметр
```

Рис. 2 – Пример №2

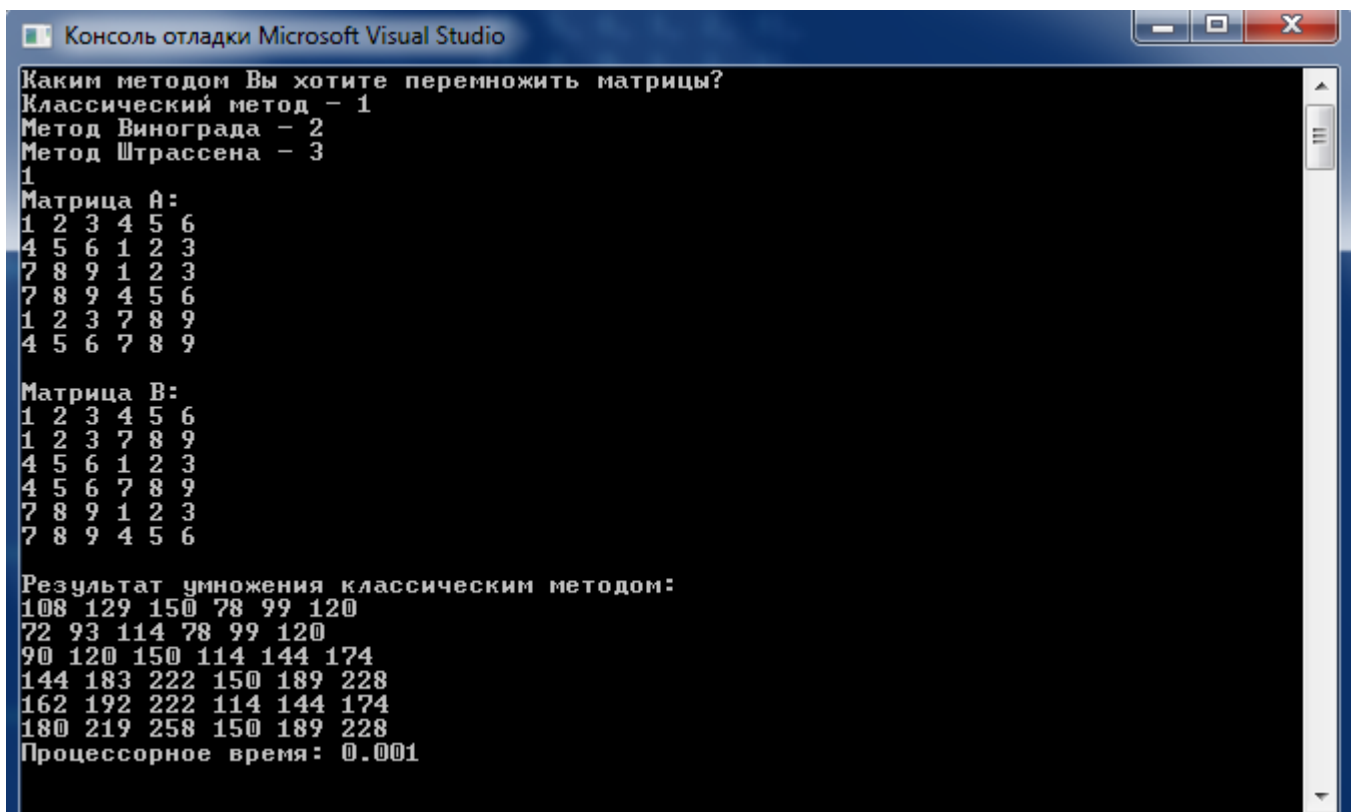
Результат работы программы умножения матриц 3×3 методом Штрассена, приведен на рис. 3.



```
Консоль отладки Microsoft Visual Studio
Каким методом Вы хотите перемножить матрицы?
Классический метод - 1
Метод Винограда - 2
Метод Штрассена - 3
3
Матрица A:
1 2 3
4 5 6
7 8 9
Матрица B:
9 8 7
6 5 4
3 2 1
Результат умножения с помощью алгоритма Штрассена:
30 24 18
84 69 54
138 114 90
Процессорное время: 0.002
C:\Users\Kononova\source\repos\ConsoleApplication4\Debug\ConsoleApplication4.exe
(процесс 4372) завершает работу с кодом 0.
Чтобы автоматически закрывать консоль при остановке отладки, установите параметр
```

Рис. 3 – Пример №3

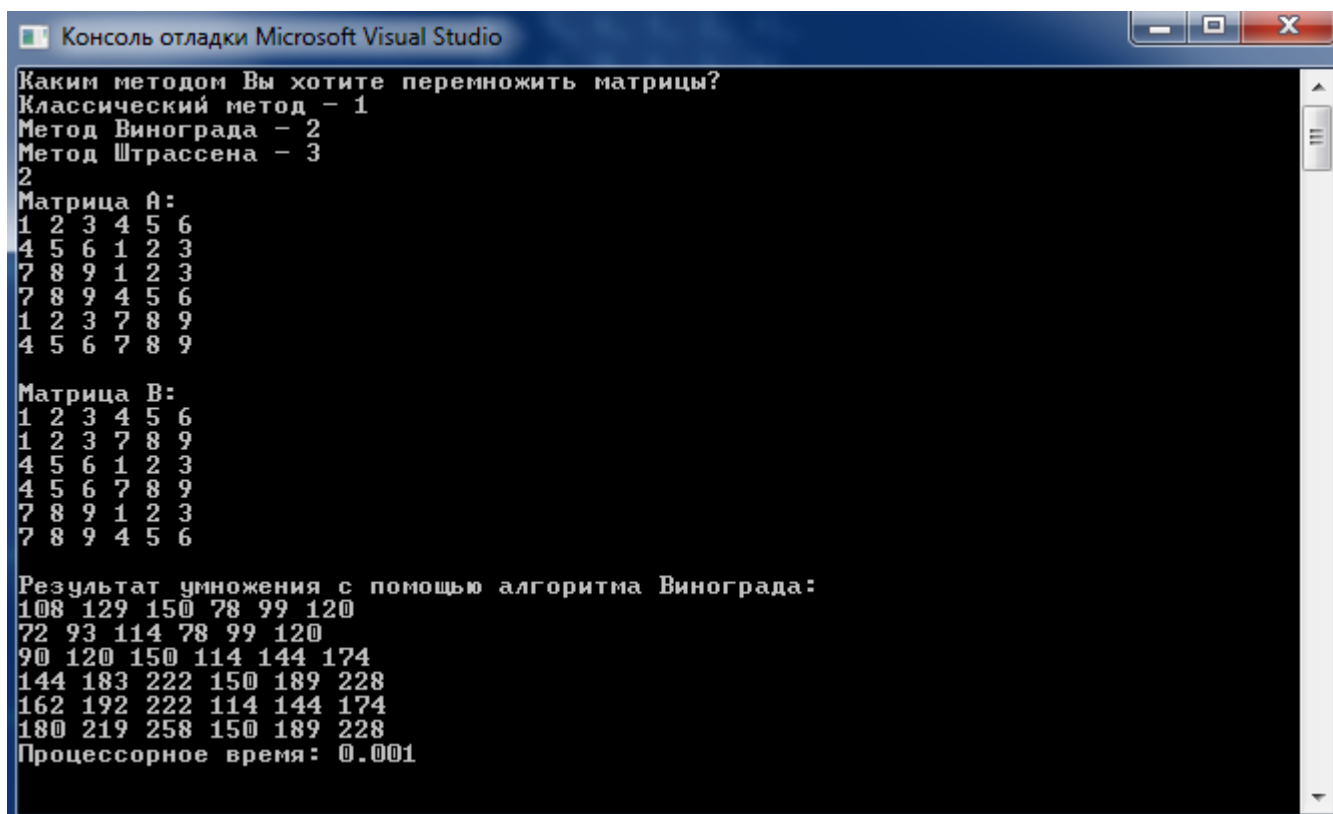
Результат работы программы умножения матриц 6×6 классическим способом, приведен на рис. 4.



```
Консоль отладки Microsoft Visual Studio
Каким методом Вы хотите перемножить матрицы?
Классический метод - 1
Метод Винограда - 2
Метод Штрассена - 3
1
Матрица A:
1 2 3 4 5 6
4 5 6 1 2 3
7 8 9 1 2 3
7 8 9 4 5 6
1 2 3 7 8 9
4 5 6 7 8 9
Матрица B:
1 2 3 4 5 6
1 2 3 7 8 9
4 5 6 1 2 3
4 5 6 7 8 9
7 8 9 1 2 3
7 8 9 4 5 6
Результат умножения классическим методом:
108 129 150 78 99 120
72 93 114 78 99 120
90 120 150 114 144 174
144 183 222 150 189 228
162 192 222 114 144 174
180 219 258 150 189 228
Процессорное время: 0.001
```

Рис. 4 – Пример №4

Результат работы программы умножения матриц 6×6 методом Винограда, приведен на рис. 5.



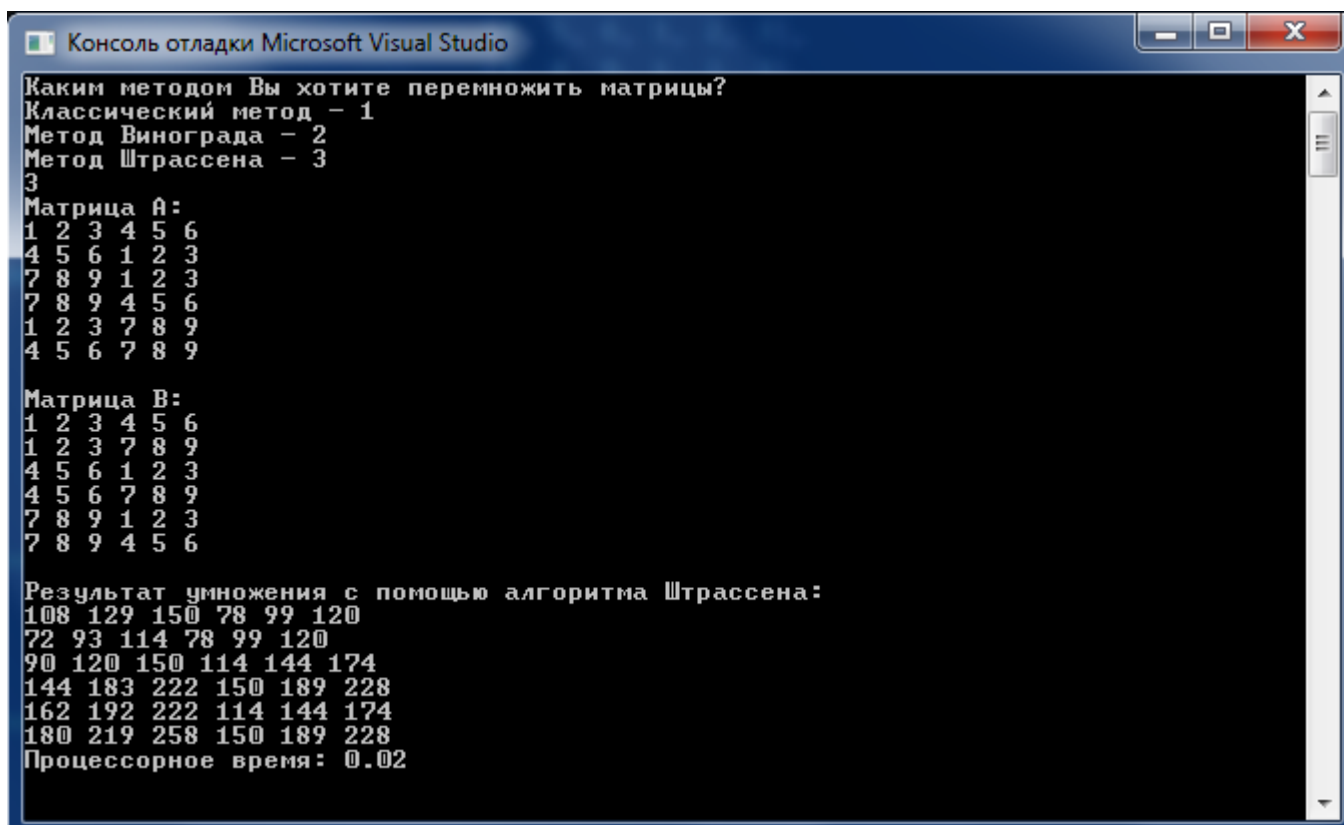
```
Консоль отладки Microsoft Visual Studio
Каким методом Вы хотите перемножить матрицы?
Классический метод - 1
Метод Винограда - 2
Метод Штрассена - 3
2
Матрица A:
1 2 3 4 5 6
4 5 6 1 2 3
7 8 9 1 2 3
7 8 9 4 5 6
1 2 3 7 8 9
4 5 6 7 8 9

Матрица B:
1 2 3 4 5 6
1 2 3 7 8 9
4 5 6 1 2 3
4 5 6 7 8 9
7 8 9 1 2 3
7 8 9 4 5 6

Результат умножения с помощью алгоритма Винограда:
108 129 150 78 99 120
72 93 114 78 99 120
90 120 150 114 144 174
144 183 222 150 189 228
162 192 222 114 144 174
180 219 258 150 189 228
Процессорное время: 0.001
```

Рис. 5 – Пример №5

Результат работы программы умножения матриц 6×6 методом Штрассена, приведен на рис. 6.



```
Консоль отладки Microsoft Visual Studio
Каким методом Вы хотите перемножить матрицы?
Классический метод - 1
Метод Винограда - 2
Метод Штрассена - 3
3
Матрица A:
1 2 3 4 5 6
4 5 6 1 2 3
7 8 9 1 2 3
7 8 9 4 5 6
1 2 3 7 8 9
4 5 6 7 8 9

Матрица B:
1 2 3 4 5 6
1 2 3 7 8 9
4 5 6 1 2 3
4 5 6 7 8 9
7 8 9 1 2 3
7 8 9 4 5 6

Результат умножения с помощью алгоритма Штрассена:
108 129 150 78 99 120
72 93 114 78 99 120
90 120 150 114 144 174
144 183 222 150 189 228
162 192 222 114 144 174
180 219 258 150 189 228
Процессорное время: 0.02
```

Рис. 6 – Пример №6

Результат работы программы умножения матриц с различными размерами классическим способом, приведен на рис. 7.

```

Консоль отладки Microsoft Visual Studio
Каким методом Вы хотите перемножить матрицы?
Классический метод - 1
Метод Винограда - 2
Метод Штрассена - 3
1
Матрица A:
1 2 3 4 5 6
4 5 6 1 2 3
7 8 9 1 2 3
7 8 9 4 5 6
1 2 3 7 8 9
4 5 6 7 8 9

Матрица B:
9 8 7
6 5 4
3 2 1

Невозможно умножить матрицы: неподходящие размеры
Результат умножения классическим методом:
Процессорное время: 0.002

C:\Users\Kononova\source\repos\ConsoleApplication4\Debug\ConsoleApplication4.exe
(процесс 19304) завершает работу с кодом 0.
Чтобы автоматически закрывать консоль при остановке отладки, установите параметр
"Сервис" -> "Параметры" -> "Отладка" -> "Автоматически закрыть консоль при оста-
новке отладки".
Чтобы закрыть это окно, нажмите любую клавишу:

```

Рис. 7 – Пример №7

Классический способ умножения матриц имеет временную сложность $O(n^3)$, где n — размерность матрицы. Это означает, что время выполнения операции умножения матриц линейно зависит от куба размера матрицы. Как результат, общее количество операций для умножения двух матриц размером $m \times p$ и $p \times n$ составляет $m \cdot p \cdot n$ умножений и $m \cdot n \cdot (p - 1)$ сложений.

Умножение матриц с помощью метода Винограда дает больший выигрыш во временной сложности. Она составляет $O(n^{2,3755})$, где n — размерность матрицы. Результаты подсчета умножений и сложений, где размер матрицы A равен $a \times b$, размер матрицы B равен $b \times c$ и $d = \frac{b}{2}$ наглядно продемонстрированы в таблице 1.

Таблица 1 – Количество операций в методе Винограда

Обработка матрицы	Умножение	Сложение
Предварительная обработка матрицы A	$a \cdot d$	$a \cdot (d - 1)$
Предварительная обработка матрицы B	$c \cdot d$	$c \cdot (d - 1)$
Вычисление матрицы R	$a \cdot c \cdot d$	$a \cdot c \cdot (2d + d + 1)$
Всего	$\frac{abc + ab + bc}{2}$	$\frac{a(b - 2) + c(b - 2) + ac(3b + 2)}{2}$
Сложение двух ненулевых матриц одинакового размера	Ненулевые матрицы M и N	Результирующая ненулевая матрица

Метод Штрассена умножает матрицы за время $O(n^{\log_2 7}) = O(n^{2.81})$, что даёт выигрыш на больших плотных матрицах. Если обозначить количество операций умножения как $M(n)$, то для обычного метода умножения матриц размера $2^n \times 2^n$ имеем $M(n) = 8^n$. Алгоритм Штрассена снижает количество операций умножения до $M(n) = 7^n$, что означает уменьшение сложности. Он достигает этого за счет уменьшения числа стандартных умножений матриц.

Исходя из вышеперечисленных фактов, можно сделать вывод, что самым эффективным алгоритмом в умножении матриц будет является алгоритм Винограда. Он умножает две матрицы за небольшое количество времени, затрачивает минимальное количество памяти и несложен в реализации.

Заключение

Цель лабораторной работы достигнута. Проведён сравнительный анализ алгоритмов умножения матриц.

В результате выполнения лабораторной работы были выполнены все задачи.

1. Описаны три алгоритма умножения матриц — классический, Винограда и Штрассена.
2. Реализованы разработанные алгоритмы.
3. Выполнено тестирование реализации разработанных алгоритмов.
4. Дана оценка сложности алгоритмов.
5. Оценена эффективность каждого алгоритма (по памяти и времени).

Список литературы

1. Основы современных алгоритмов 2-е дополненное издание / Дж. Макконелл // под ред. С.Кландо, доп. М.В.Ульянова. — Москва: ТЕХНОСФЕРА, 2004. — 370 с.